# TechNest – A Modular Front-End eCommerce Web Application

Muhammad Saleem

L00196822

*PROJ_IT805 -*
*LY_ICSWD_B:*
*Project (2025/26)* 2026

Ollscoil Teicneolaíochta an Atlantaigh Dún na nGall
Atlantic Technological University Donegal

Department of Computing, ATU Donegal, Port Road, Letterkenny, Co. Donegal, Ireland.

# TechNest – A Modular Front-End eCommerce Web Application

Author: Muhammad Saleem - L00196822

Supervised by: Lusungu Mwasinga, Pauric Dawson

Submitted to Atlantic Technological University

*Arna chur isteach chuig Ollscoil Teicneolaiochta an Atlantaigh* April 2026

## Declaration

I hereby certify that the material, which l now submit for assessment on the programmes of study leading to the award of BSc (Hons) in Contemporary Software Development in 2026, is entirely my own work and has not been taken from the work of others except to the extent that such work has been cited and acknowledged within the text of my own work. No portion of the work contained in this thesis has been submitted in support of an application for another degree or qualification to this or any other institution. I understand that it is my responsibility to ensure that I have adhered to ATU's rules and regulations.

I hereby certify that the material on which I have relied on for the purpose of my assessment is not deemed as personal data under the GDPR Regulations. Personal data is any data from living people that can be identified. Any personal data used for the purpose of my assessment has been pseudonymised and the data set and identifiers are not held by ATU. Alternatively, personal data has been anonymised in line with the Data Protection Commissioners Guidelines on Anonymisation.

I consent that my work will be held for the purposes of education assistance to future students and will be shared on the ATU Donegal (Computing) website (atucomputingdonegal.com) and Research THEA website (https://research.thea.ie/). I understand that documents once uploaded onto the website can be viewed throughout the world and not just in the Ireland. Consent can be withdrawn for the publishing of material online by emailing Jade Lyons; Head of Department at Jade.Lyons@atu.ie to remove items from the ATU Donegal Computing website and by email emailing Denise McCaul; Systems Librarian at denise.mccaul@atu.ie to remove items from the Research THEA website. Material will continue to appear in printed formats once published and as websites are public medium, ATU cannot guarantee that the material has not been saved or downloaded.

| Signature of Candidate | Date |
| --- | --- |
| Muhammad Saleem | 10th April 2026 |

# Acknowledgements

I would like to thank Dr. Lusungu Mwasinga and Mr. Pauric Dawson for their support and guidance during this project.

# Abstract

This This project involved the design and development of a front-end eCommerce web application called TechNest. The application was created for a fictional electronics and gadgets retailer and runs entirely in the browser using HTML5, CSS3, Bootstrap 5 and JavaScript (ES6+). No server or database is used, as all functionality and data management are handled on the client side.

The project aimed to address the lack of structured and maintainable code often seen in introductory front-end applications. Many beginner-level projects are developed using a procedural approach with limited organisation. TechNest demonstrates how Object-Oriented Programming principles and a modular structure can be applied effectively in a client-side environment to improve code clarity and maintainability.

The final application allows users to browse products, filter and search the catalogue, view product details, manage a shopping basket and complete a checkout process. Basket data is stored using the browser's localStorage, ensuring persistence across page navigation. The project was developed over a 10-week period, with version control managed using Git and GitHub throughout.

The project highlights how modern front-end technologies can be used to simulate real-world application behaviour without requiring a backend system. It also demonstrates the importance of structured development, testing, and iterative refinement in producing reliable and maintainable software solutions.

# Table of Contents

Declaration ..... iii

Acknowledgements ..... iv

Abstract ..... v

Table of Contents ..... vi

2. Design ..... 1

**2.1. Introduction** ..... 1

**2.2. Project Requirements (System Requirements)** ..... 1

2.2.1. Functional Requirements ..... 1

2.2.2. Non-Functional Requirements ..... 2

2.2.3. User Stories ..... 3

**2.3. Justification of Technology and Software Used** ..... 4

2.3.1 HTML5 ..... 4

**2.3.2 CSS3** ..... 4

**2.3.3 Bootstrap 5** ..... 4

**2.3.4 JavaScript ES6+** ..... 5

**2.3.5 localStorage API** ..... 5

**2.3.6 Visual Studio Code** ..... 6

**2.3.7 Git and GitHub** ..... 6

**2.3.8 Chrome Developer Tools** ..... 6

2.4 Diagrams to Support the Project ..... 7

**2.4.1 Layered Architecture Diagram** ..... 7

**2.4.2 IPO Diagram** ..... 8

**2.4.3 Folder Structure Diagram** ..... 9

**2.4.4 Page Flow Diagram** ..... 10

**2.4.5 Class Diagram** ..... 13

**2.4.6 Wireframes** ..... 14

2.5 Summary ..... 15

References ..... a

# 2. Design

## 2.1. Introduction

This chapter covers the design decisions made for TechNest. It includes the requirements gathered before development started, the technology choices made and the reasons behind them, and a set of diagrams to show how the application is structured. The chapter also covers the user stories used to guide what features needed to be built.

Most of the design work was done in weeks one and two of the project. The requirements, user stories and planning diagrams including the wireframes, folder structure diagram, page flow diagram and IPO diagram were all produced in week one before any code was written. The structural diagrams, namely the layered architecture diagram and class diagram, were drawn up in week two. The module structure and class design had already been planned in week one but the visual diagrams were produced once development was about to begin. Having this plan in place before development started made it much easier to stay on track and to know what needed to be built at each stage.

## 2.2. Project Requirements (System Requirements)

The following project requirements define the functional and non-functional behaviour expected from the TechNest system. Before starting to build TechNest, a set of requirements was defined to make clear exactly what the application needed to do. These are split into functional requirements, which describe what the system should do, and non-functional requirements, which describe how well it should do it.

### 2.2.1. Functional Requirements

The following functional requirements were identified based on the aims and objectives set out in Chapter 1. Each one describes a specific piece of behaviour the application must support.

- FR1 – The application shall display a homepage with featured products and navigation links to other pages.

- FR2 – The application shall render a product catalogue dynamically from JavaScript data objects using DOM manipulation.
- FR3 – The user shall be able to filter products by category.
- FR4 – The user shall be able to search for products using a keyword search field.
- FR5 – The user shall be able to sort the product catalogue by price or name.
- FR6 – Each product shall have its own detail page, loaded using URL parameter handling.
- FR7 – The user shall be able to add products to a basket.
- FR8 – The user shall be able to update item quantities in the basket.
- FR9 – The user shall be able to remove items from the basket.
- FR10 – The basket total shall update automatically when items are added, updated or removed.
- FR11 – Basket contents shall persist across page navigation using localStorage.
- FR12 – The user shall be able to complete a checkout form with fields for personal and delivery details.
- FR13 – The checkout form shall validate all required fields on the client side before submission.
- FR14 – A confirmation message shall be displayed after a successful checkout.

### 2.2.2. Non-Functional Requirements

Alongside the functional requirements, a number of non-functional requirements were also defined. These are about quality and performance rather than specific features.

NFR1 – The application shall be responsive and usable on desktop, tablet and mobile screen sizes.

NFR2 – Pages shall load quickly without any server-side requests. All logic runs in the browser.

NFR3 – JavaScript code shall be organised into separate modules with clear separation between data, logic and presentation.

NFR4 – The codebase shall use ES6 class syntax for all main data and logic components.

NFR5 – The application shall work in all modern browsers (Chrome, Firefox, Edge, Safari).

NFR6 – All source code shall be version controlled using Git with regular commits pushed to GitHub.

### 2.2.3. User Stories

User stories were used to help think about the requirements from the perspective of someone actually using the site. They helped make sure each feature had a clear purpose and was not just added for the sake of it.

- As a shopper, I want to browse all available products so that I can see what the store sells.
- As a shopper, I want to filter products by category so that I can find items relevant to me more quickly.
- As a shopper, I want to search for a product by name so that I do not have to scroll through everything.
- As a shopper, I want to click on a product and see more details about it before deciding to buy.
- As a shopper, I want to add items to a basket so that I can purchase multiple products at once.
- As a shopper, I want my basket to save when I move between pages so I do not lose what I have added.
- As a shopper, I want to be able to update the quantity of an item or remove it from my basket.
- As a shopper, I want to fill in my details at checkout and get a confirmation that my order went through.

## 2.3. Justification of Technology and Software Used

Every technology used in TechNest was chosen deliberately. This section explains what was used and why, rather than just listing the tools. The decision to keep everything client-side had a big influence on which technologies made sense.

### 2.3.1 HTML5

HTML5 was used to build the structure of every page in the application. One of the reasons HTML5 was chosen over older versions of HTML is its support for semantic elements such as <header>, <nav>, <main>, <section> and <footer>. Using these makes the code much easier to read and understand because the structure tells you what each part of the page is supposed to do, without having to rely on class names or comments to explain it.

HTML5 also supports form validation attributes like required, minlength and pattern, which were used on the checkout page to prevent blank or invalid submissions without needing to write lots of extra JavaScript. This kept the checkout validation code simpler and more reliable.

### 2.3.2 CSS3

CSS3 was used for all custom styling across the application. While Bootstrap handles most of the layout and responsive grid, custom CSS was needed to style things like the product cards, the basket summary panel, and the navbar branding. CSS3 features such as flexbox, custom properties (variables) and transitions were used to keep the styling clean and consistent across pages.

One of the reasons CSS was kept separate from Bootstrap rather than overriding Bootstrap classes directly was to avoid making the code hard to debug. By writing custom styles in a dedicated stylesheet, it was much easier to find and change specific styles during development.

### 2.3.3 Bootstrap 5

Bootstrap 5 was chosen as the front-end framework for TechNest because it makes building responsive layouts much faster and more reliable than writing all the grid and breakpoint

logic from scratch. The grid system and utility classes meant the product listing page, basket and checkout all worked across different screen sizes with minimal extra effort.

Bootstrap 5 was specifically chosen over Bootstrap 4 because it removed its jQuery dependency entirely (Bootstrap, 2021). Since TechNest does not use jQuery anywhere, using Bootstrap 5 kept the project consistent and avoided pulling in an unnecessary library. Bootstrap components such as the navbar, cards, buttons and modal were all used across the application.

### 2.3.4 JavaScript ES6+

JavaScript ES6+ is the core language used to build all application logic in TechNest. ES6 was chosen over older JavaScript versions because of the features it introduced that make code much easier to organise and maintain. Class syntax was used to define the Product, Basket and StorageManager objects, which was central to the OOP approach taken in this project.

Arrow functions, template literals and destructuring were used throughout to make the code shorter and easier to read. The native module system (import/export) was used to split the application into separate files with clear responsibilities. This meant each file had one job to do, which made debugging a lot easier when things went wrong.

A server-side language such as PHP or Node.js was considered early on but was ruled out because the project specification focused on front-end development. Keeping everything in JavaScript also meant the entire application could be run from a local folder or hosted as a static site without needing a web server.

### 2.3.5 localStorage API

The browser's localStorage API was chosen to handle data persistence in TechNest. Since there is no back-end database, localStorage provided a straightforward way to save and retrieve basket data across pages. When a user adds a product to their basket on the products page and then navigates to the basket page, the items are still there because they were saved to localStorage before the page changed.

sessionStorage was considered as an alternative, but it clears data when the browser tab is closed, which would not be a good user experience. localStorage keeps the data until it is explicitly cleared, which felt like the right behaviour for a shopping basket. The data is stored as JSON strings, which are serialised and deserialised using JSON.stringify() and JSON.parse() inside the StorageManager class.

### 2.3.6 Visual Studio Code

Visual Studio Code was used as the code editor throughout the project. It was already familiar from previous modules, which meant less time was wasted learning a new tool. Extensions such as Live Server (for running pages locally in the browser with auto-reload) and Prettier (for code formatting) were particularly useful during development. The built-in Git integration also made committing and pushing to GitHub quick and easy without needing to use the command line every time.

### 2.3.7 Git and GitHub

Git was used for version control throughout the project, and all commits were pushed to a GitHub repository. Regular commits were made at the end of each significant piece of work so that the development history is clear and any mistakes could be rolled back if needed. GitHub also acts as a backup, which was important given the 10-week timeline – losing work at any point would have been very difficult to recover from.

### 2.3.8 Chrome Developer Tools

Chrome Developer Tools was used extensively during development for debugging JavaScript, inspecting the DOM, checking localStorage values, and testing the responsive layout at different screen sizes. The console was particularly useful when tracing errors in the basket logic, and the Network tab was occasionally used to check that no unintended network requests were being made, since TechNest is supposed to be entirely client-side.

## 2.4 Diagrams to Support the Project

This section contains six diagrams produced during the design phase of the project. Together they cover how the application is structured, how data flows through it, how files are organised, how pages connect, how the ES6 classes relate to each other, and what each page was planned to look like before any code was written. All diagrams were created in week one and used as a reference throughout development.

### 2.4.1 Layered Architecture Diagram

Figure 2.1 shows how TechNest is organised as a layered system. There are four layers stacked from top to bottom. The Presentation layer at the top contains all five HTML pages – index.html, products.html, product-detail.html, basket.html and checkout.html – along with the shared style.css file and Bootstrap 5 which handle styling and responsive layout across every page.

Below that sits the Logic layer, which contains the page-specific JavaScript modules in the js/pages/ folder. Each module is responsible for one page – for example, products.js handles rendering the catalogue, filtering and search, while basket.js handles displaying basket contents and processing quantity changes. Also in this layer are the Basket.js and Product.js class files, which contain the core application logic.

The Data layer holds StorageManager.js, which handles reading and writing to localStorage, and the products.js data file inside the data/ folder, which is a static array of product objects and acts as the single source of truth for all product information. There is no database or server in this layer – this is intentional and is highlighted in the diagram as a deliberate design choice.

The Browser Runtime layer at the bottom shows the three browser APIs the application depends on: the localStorage API for persisting basket data, the DOM API for dynamically rendering products and basket contents, and the URL/Location API for reading URL parameters on the product detail page. Everything runs natively in the browser with no external dependencies beyond Bootstrap.

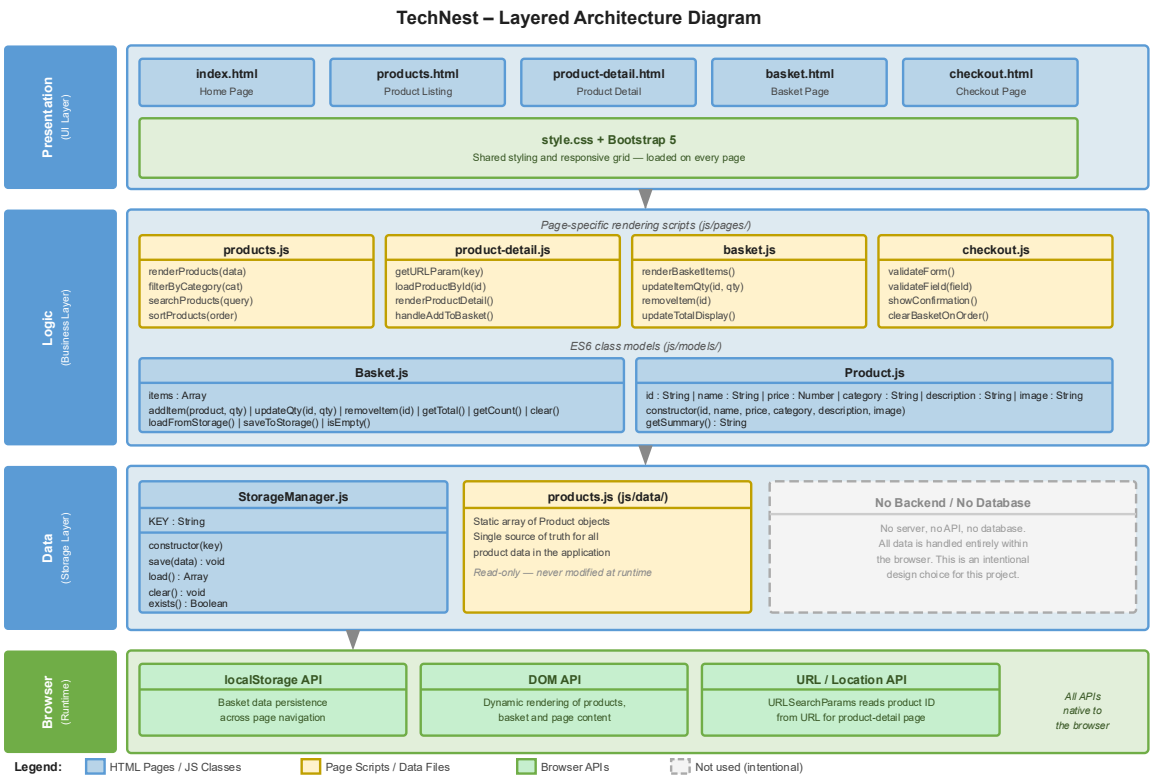


*Figure 2.1: TechNest Layered Architecture Diagram*

### 2.4.2 IPO Diagram

Figure 2.2 is an IPO (Input – Process – Output) diagram. Where the architecture diagram shows how the code is structured, the IPO diagram shows what actually happens when a user interacts with the application. It covers six key interactions: product browsing, search and filter, viewing a product detail, adding to basket, managing the basket, and completing checkout.

For product browsing, the input is the user opening the products page. The process is the products data file being read and each product object being loaded into the page via DOM manipulation. The output is the product cards appearing on screen with name, image, price and category.

For the basket, when the user selects a quantity and clicks Add to Basket, the Basket class checks whether the product is already in the basket. If it is, the quantity is updated; if not, a new entry is added. The basket is then saved to localStorage and the navbar item count updates. At checkout, all form fields are validated and if everything is valid the order is

processed, the basket is cleared from localStorage and a confirmation message is shown on screen.

This diagram was useful during planning because it made it clear which JavaScript functions and class methods would need to exist before any code was written.

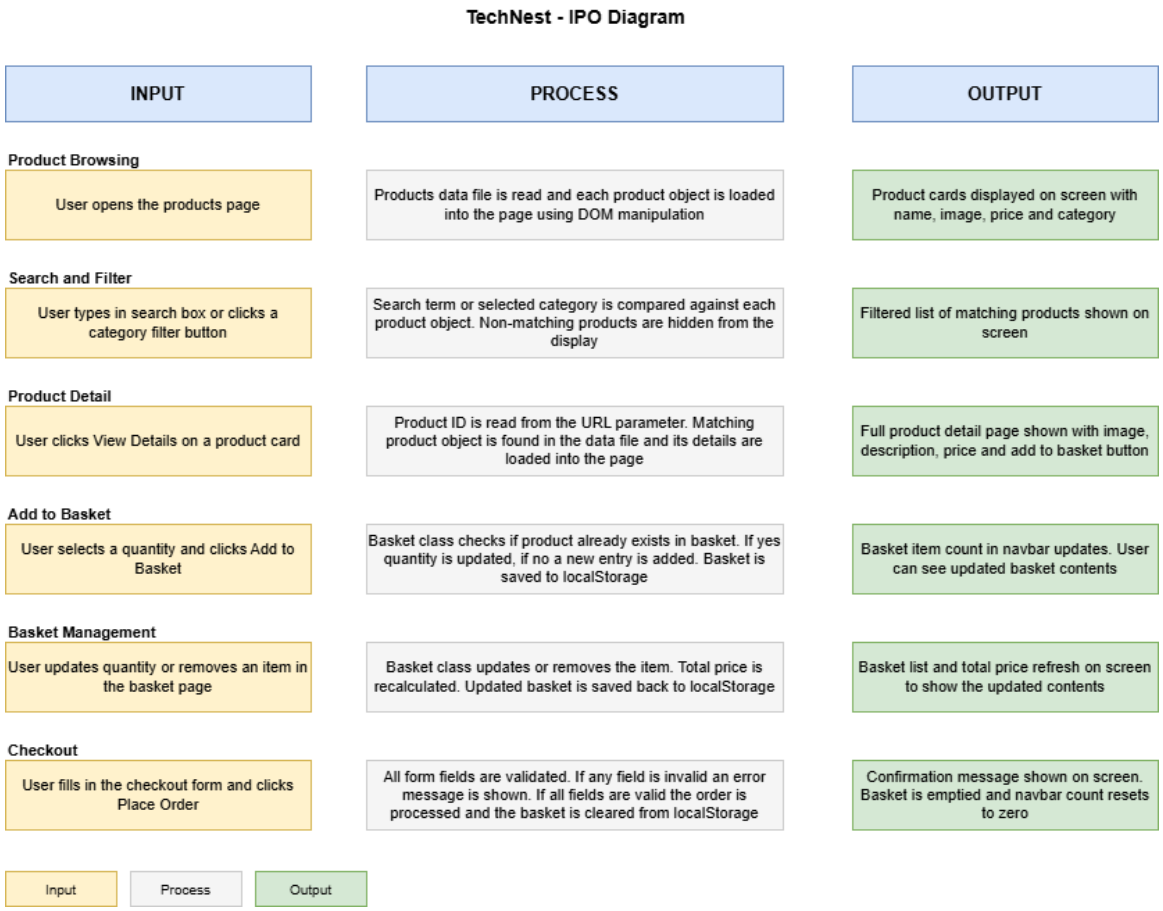


*Figure 2.2: TechNest IPO Diagram*

### 2.4.3 Folder Structure Diagram

Figure 2.3 shows the folder and file structure of the TechNest project. The root of the project is the technest-webapp/ folder, which contains all five HTML files directly at the top level. From there, four sub-folders branch off: css/, js/, assets/ and docs/.

The css/ folder holds the single stylesheet, style.css. The js/ folder is split into three sub-folders: data/ contains the products.js data file, models/ contains the three class files (Product.js, Basket.js and Storage.js), and pages/ holds the page-specific rendering scripts.

The assets/ folder contains an images/ sub-folder for product images, and the docs/ folder holds planning documents including a diagrams/ sub-folder.

This structure was agreed at the start of the project and kept consistent all the way through. Separating data, models and page scripts into their own folders was a deliberate decision to reflect the separation of concerns principle. It meant that if something needed changing in the basket logic, it was always clear that Basket.js in the models/ folder was the right place to look, rather than having to search through a flat list of files.

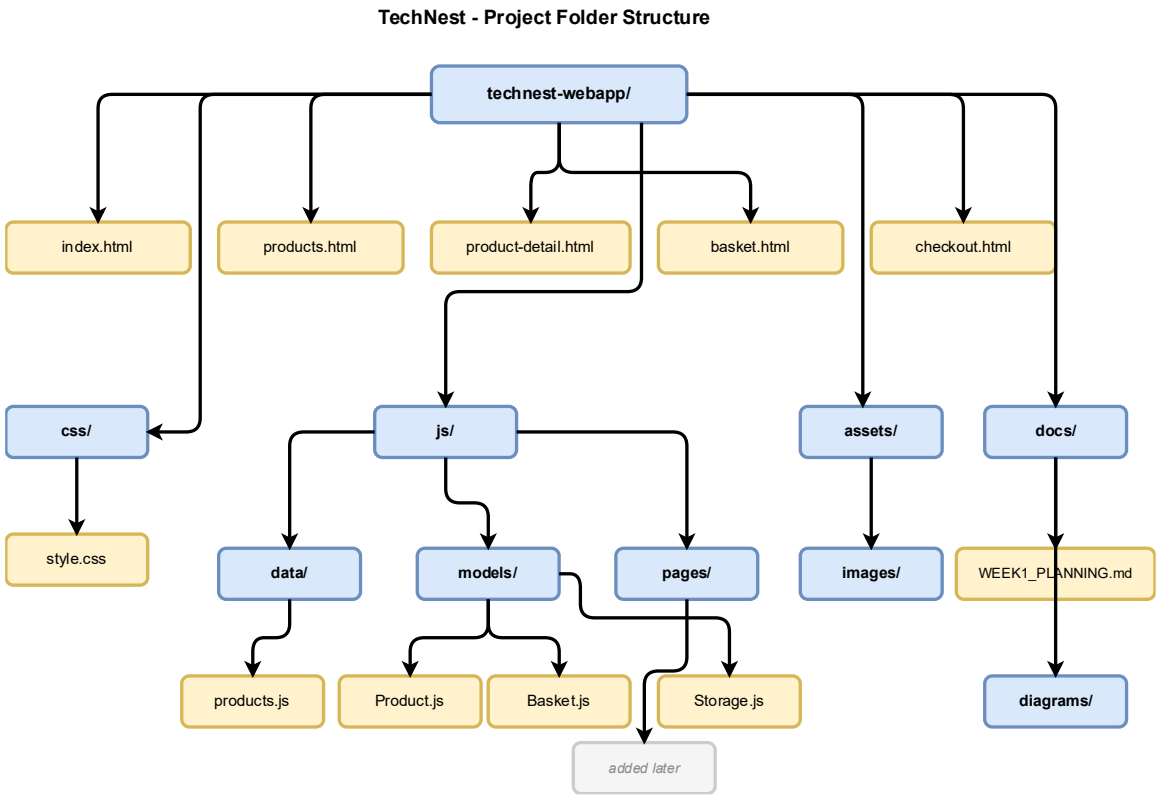


*Figure 2.3: TechNest Folder Structure Diagram*

### 2.4.4 Page Flow Diagram

Figure 2.4 shows the user page flow through TechNest. The diagram uses blue boxes for pages, a green box for the end state, and dashed lines for optional routes. A user starts on the Home Page (index.html) and clicks the Shop Now button to reach the Products Page (products.html). From there they can click View Details on any product card to go to the Product Detail Page (product-detail.html).

From the product detail page, clicking Add to Basket takes the user to the Basket Page (basket.html). The dashed lines show that a user can also go back to products from the product detail page, or continue shopping from the basket back to the products page at any point. Once the user is happy with their basket, they click Proceed to Checkout which loads the Checkout Page (checkout.html). Clicking Place Order after filling in the form triggers the confirmation message, which is the final state shown in green.

The note on the left of the diagram is worth mentioning – the navbar is present on every single page, which means the user can always jump directly to the Basket or the Products page regardless of where they are. This was an important usability decision made during design.

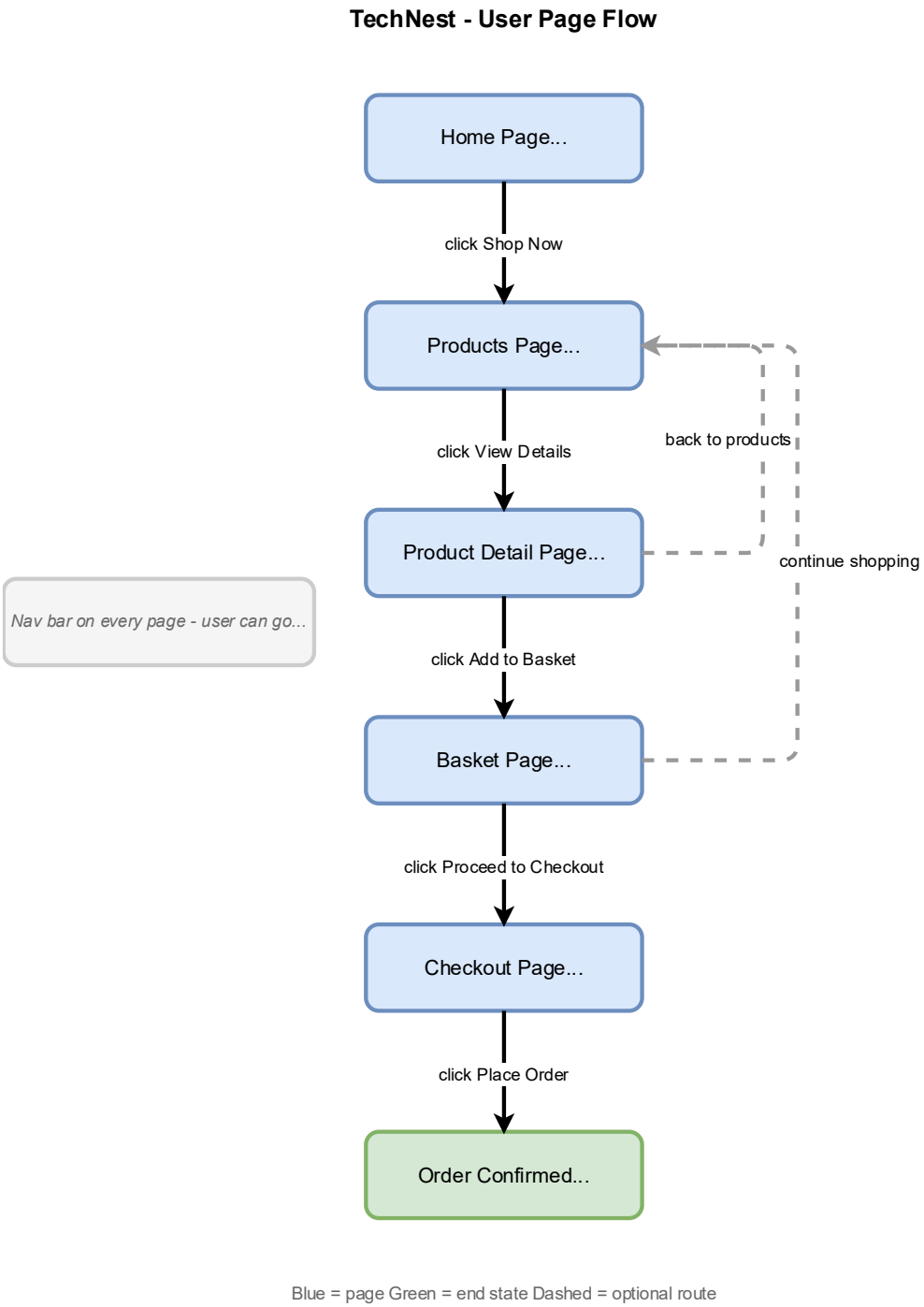


*Figure 2.4: TechNest Page Flow Diagram*

### 2.4.5 Class Diagram

Figure 2.5 shows the three ES6 classes used in TechNest and how they relate to each other. The Product class holds six properties (id, name, price, category, description and image) and has a constructor and a getSummary() method. It is a straightforward data class with no dependencies on the other classes.

The Basket class is the most involved of the three. It holds an items array and has methods for adding items (addItem), updating quantity (updateQty), removing items (removeItem), getting the running total (getTotal), getting the item count (getCount), clearing the basket (clear), and loading and saving to storage (loadFromStorage and saveToStorage). The solid arrow from Product to Basket shows that Product objects are stored inside the Basket's items array.

StorageManager handles all interaction with the browser's localStorage. It has a KEY property to identify where data is stored, and five methods: constructor(key) to initialise the instance, save() to write data, load() to retrieve it, clear() to wipe it, and exists() to check whether any saved data is present. The dashed arrow from Basket to StorageManager shows a dependency relationship, the Basket class calls StorageManager to persist and retrieve its state. The Page Scripts note box at the bottom shows that the page-specific JavaScript files interact with both the Product data and the Basket class to render the UI.

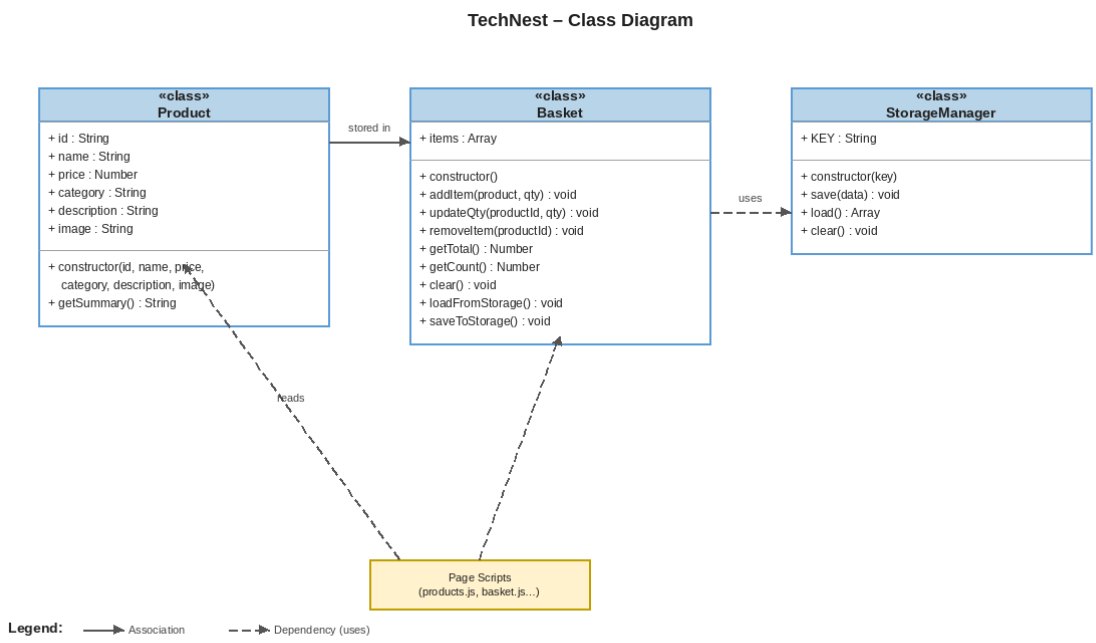


*Figure 2.5: TechNest Class Diagram*

### 2.4.6 Wireframes

Figure 2.6 shows the week one wireframe sketches for all five pages. These were deliberately kept simple, the purpose was to decide on layout and structure, not colour or visual style. They were drawn before any HTML was written and served as a reference during the early weeks of development.

The Home Page wireframe shows a navbar with logo, Products link and basket count, followed by a hero section with a short description and a Shop Now button, then a category filter row and a row of three featured product cards below it. The Products Page wireframe shows the same navbar, a search box with a sort-by-price option, a row of category filter buttons, and a grid of product cards each showing image, name, price and a View Details button.

The Product Detail wireframe shows a two-column layout with the product image on the left and the product name, category badge, description, price, quantity selector and Add to Basket button on the right. The Basket Page shows a list of items on the left with image,

name, price, quantity and a remove option, alongside an order summary panel on the right with subtotal and a Proceed to Checkout button. The Checkout Page wireframe shows a form split into Personal Details, Delivery Address and Payment Details sections, with an order summary on the right, a Place Order button at the bottom, and a note that submitting shows a confirmation message and clears the basket.

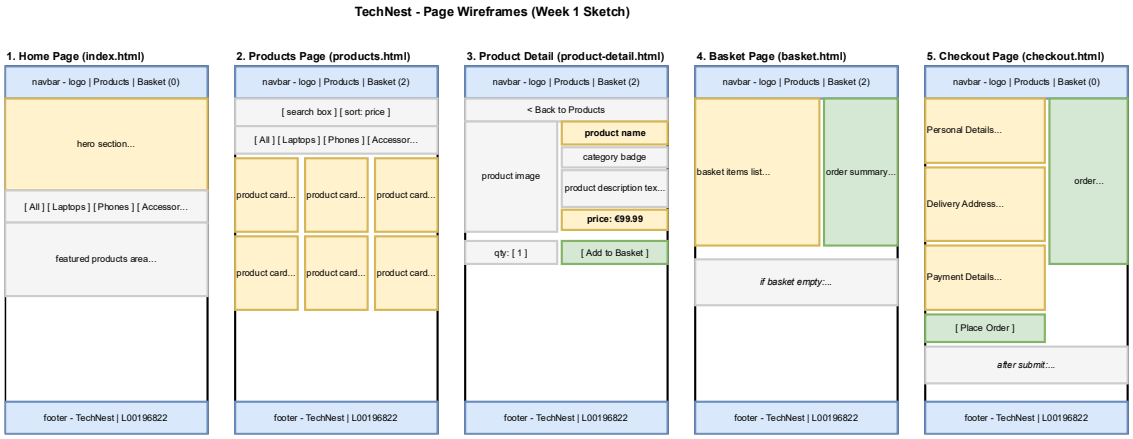


*Figure 2.6: TechNest Page Wireframes*

## 2.5 Summary

This chapter covered the full design work done before development started. The requirements define exactly what TechNest needs to do and how well it should do it. The technology choices were all made deliberately, with the decision to stay entirely client-side being the most significant one as it shaped everything else. The six diagrams in section 2.4 give a complete picture of the application from different angles such as structure, behaviour, file organisation, navigation, class design and page layout. The next chapter explains how all of this was turned into working code.

.

## References

Bootstrap (2021) Bootstrap 5.0 Documentation. Available at: https://getbootstrap.com/docs/5.0 (Accessed: 3 March 2026).